The Journal of Open Source Software

Snowdrop: Python Package for DSGE Modeling

Alexei Goumilevski [1] **and James Otterson** [1]

1 International Monetary Fund

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Nikoleta Glynatsi

Submitted: 29 January 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

At its core, `Snowdrop` is a robust and versatile Python package designed for the analysis of macroeconomic *Dynamic Stochastic General Equilibrium* (*DSGE*) models. In its entirety, this package offers an extensive framework for the study of various related economic models, including *New Keynesian* models, *Real Business Cycle* models, *Gap* models, and *Overlapping Generations* models. `Snowdrop` equips researchers with essential tools to address the fundamental requirements of these models, encompassing estimation, simulation, and forecasting processes. In particular, the package employs robust and efficient solution techniques to solve both linear and nonlinear perfect foresight models based on the rational expectations hypothesis, which is a critical need for many *DSGE* models.

Statement of need

DSGE models are a mainstay class of models employed by Central Banks around the world, informing key country monetary policy decisions (Botman et al., 2007), (Smets et al., 2010), (Del Negro et al., 2013), (Yagihashi, 2020). These models capture the dynamic evolution of economic variables influenced by agents who respond to anticipated future outcomes in the present, necessitating the combined use of specialized techniques that are not readily availabel even in the extensive list of Python's scientific modeling packages (Fernández-Villaverde & Guerrón-Quintana, 2021). Currently, the two primary DSGE modeling toolboxes, DYNARE and IRIS are comprehensive toolsets that offer an user-friendly infrastructure with support to all stages of model development. These, and similar, applications, however, are either commercial, or rely on commercial software to run, and hence require expensive licensing costs. There is no integrated software package to our knowledge that is both flexible to handle a wide class of models with all required software to run the models available for free under the *GNU General Public Licensing* agreements. This Framework, built entirely on Python, is intended to fill that void.

Highlights

- `Snowdrop` is a Python package that only uses open source libraries listed in the pypi repository.
- This package is platform neutral and can be run on Windows, Linux, Unix, and Mac machines.
- `Snowdrop` models can be written in user-friendly *YAML* format, pure Python scripts, or in a combination of both.
- Non-linear equations are solved iteratively via Newton's method. `Snowdrop` implements the *ABLR* stacked matrices and *LBJ* (Juillard et al., 1998) forward-backward substitution method to solve such systems. Linear models are solved with *Binder Pesaran's* method, *Anderson and More's* method and two generalized *Schur's* method that reproduce calculations employed in *Dynare* and *Iris*.

DRAFT

- Several desirable computational techniques for *DSGE* models are implemented in `Snowdrop`, including:
  - Non-linear models can be run with time dependents parameters
  - Goodness of fit of model data can be checked via the *Bayesian* approach to the maximization of likelihood functions.
  - Model parameters can be sampled via the *Markov Chain Monte Carlo* affine invariant ensemble sampler algorithm of Jonathan Goodman and an adaptive Metropolis-Hasting's algorithms of Paul Miles. The former algorithm is useful for sampling badly scaled distributions of parameters. The later algorithm employs adaptive Metropolis methods that incorporate delayed rejection to stimulate samples' states mixing.
- Finally, `Snowdrop` streamlines the model production process by aiding users with the plotting and model reporting and storage process.

Examples of model files and python code

The simplest way to write a `Snowdrop` model, is by specifing it via an *YAML* file in a manner that is familiar to *DYNARE* and *IRIS* users. Overall, the quickest way to run a model involves the following steps:

1. Create or modify existing *YAML model file* in models folder.
2. Open *src/tests/test_toy_models.py* file and set *fname* to the name of this model file.
3. Run the python script to get the desired simulations.

For example, the following specify a simple monetary policy model with lagged variables.

Monetary policy model file

```yaml
name:  Monetary policy model example
symbols:
  variables: [PDOT,RR,RS,Y]
  exogenous: [ers]
  shocks: [ey]
  parameters: [g,p1,p2,p3,p4,p5,p6,p7]
  equations:
   - PDOT=p1*PDOT(+1)+(1-p1)*PDOT(-1)+p2*(g^2/(g-Y)-g)+p3*(g^2/(g-Y(-1))-g)
   - RR=RS-p1*PDOT(+1)-(1-p1)*PDOT(-1)
   - RS=p4*PDOT+Y+ers
   - Y=p5*Y(-1)-p6*RR-p7*RR(-1)+ey
  calibration:
   #Parameters
   g: 0.049
   #Set time varying parameters; the last value will be used for the rest of this ar
   p1: 0.414 #[0.4,0.5,0.6]
   std: 0.02
options:
   T: 14
   periods: [1]
   shock_values: [std]
```

Imposing shocks

```python
# Create model object
from snowdrop.src import driver
model = driver.importModel(model_file_path)
```

DRAFT

```
# Set shocks
model.options["periods"] = [1]
model.options["shock_values"] = [0.02]
# Define list of variables for which decomposition plots are produced
decomp = ['PDOT','RR','RS','Y']
# Run forecast
y, dates = driver.run(model=model, decomp_variables=decomp, Plot=True)
```

Anticipated, unanticipated shocks, and judgmental ajustments

```
from snowdrop.src.driver import run
# Set shock for gap of output to 1% at period 3
d = {"SHK_L_GDP_GAP": [(3,1)]}
model.setShocks(d)
model.anticipate = True
# Impose judgments
date_range = pandas.date_range(start, end, freq="QS")
m = {"L_GDP_GAP": pandas.Series([-1.0, -1.0, -1.0], date_range)}
shocks_names  = ["SHK_L_GDP_GAP"]
# Endogenize shock and exogenize output gap endogenous variable
model.swap(m, shocks_names)
# Run simulations
y, dates = driver.run(model)
```

Status

This toolkit provides users with an integrated Framework to input their models, import data, perform the desired computational tasks (solve, simulate, calibrate or estimate) and obtain well formatted post process output in the form of tables, graphs etc. (Goumilevski et al., 2021). It has been applied for several cases including study of macroeconomic effects of monetary policy, estimation of Peter's Ireland model (Ireland, 2004), and forecast of economic effects of COVID-19 virus, to name a few. Figure below illustrates forecast of inflation, nominal and real interest rates, and output gap to output shock of 2% imposed at period 1 and revision of monetary policy rate of 3% imposed at period 4.

DRAFT

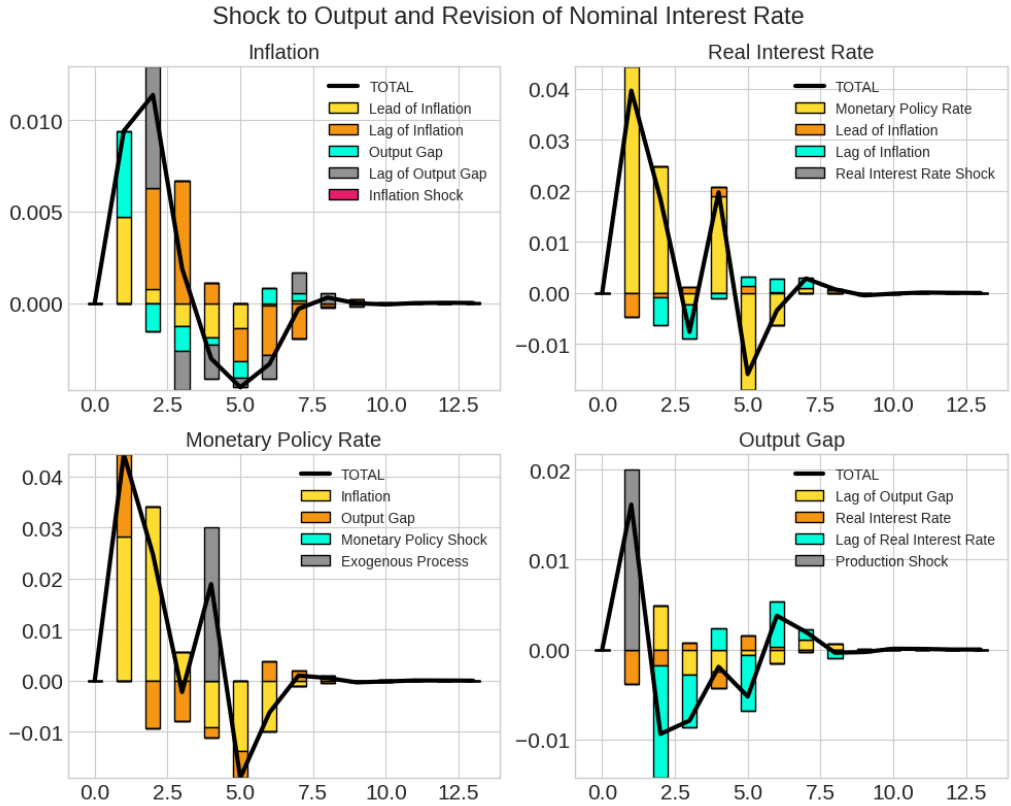

Figure 1: Monetary Policy Example

Another example illstrates economic effects of pandemic. We used Eichenbaum-Rebelo-Trabandt (*ERT*) model (Eichenbaum et al., 2020) which embeds epidemiological concepts into *New Keynesian* modelling framework. We assumed that there two strains of pathogens and emplyed Suspected-Infected-Recovered (*SIR*) epideomiological model. These epideomiological equations were plugged in into *ERT* model consisting of sixty-four equations of macroeconomic variables of sticky and flexible price economies. The macroeconomic variables of these two economies were linked thru Taylor rule equation for policy interest rate. Model is highly non-linear and is solved by using a homotopy method where parameters are adjusted step-by-step. We assumed that the government containment measures were more lenient during the second strain of virus compared to the first one. Figures 2 and 3 illustrate forecast of virus transmission and deviations of macroeconomic variables from their steady state.

DRAFT

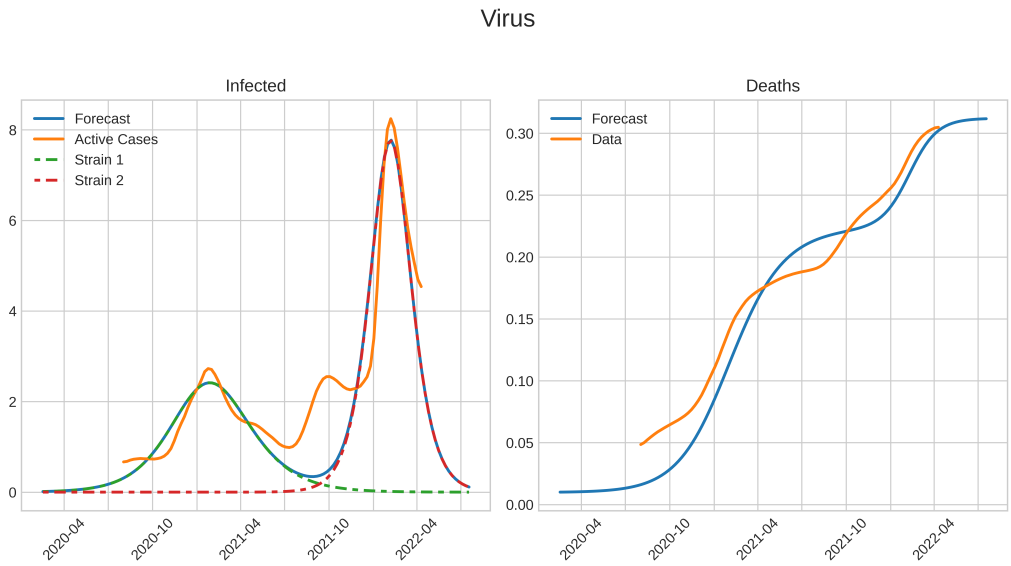

Figure 2: Epidemic Forecast

DRAFT

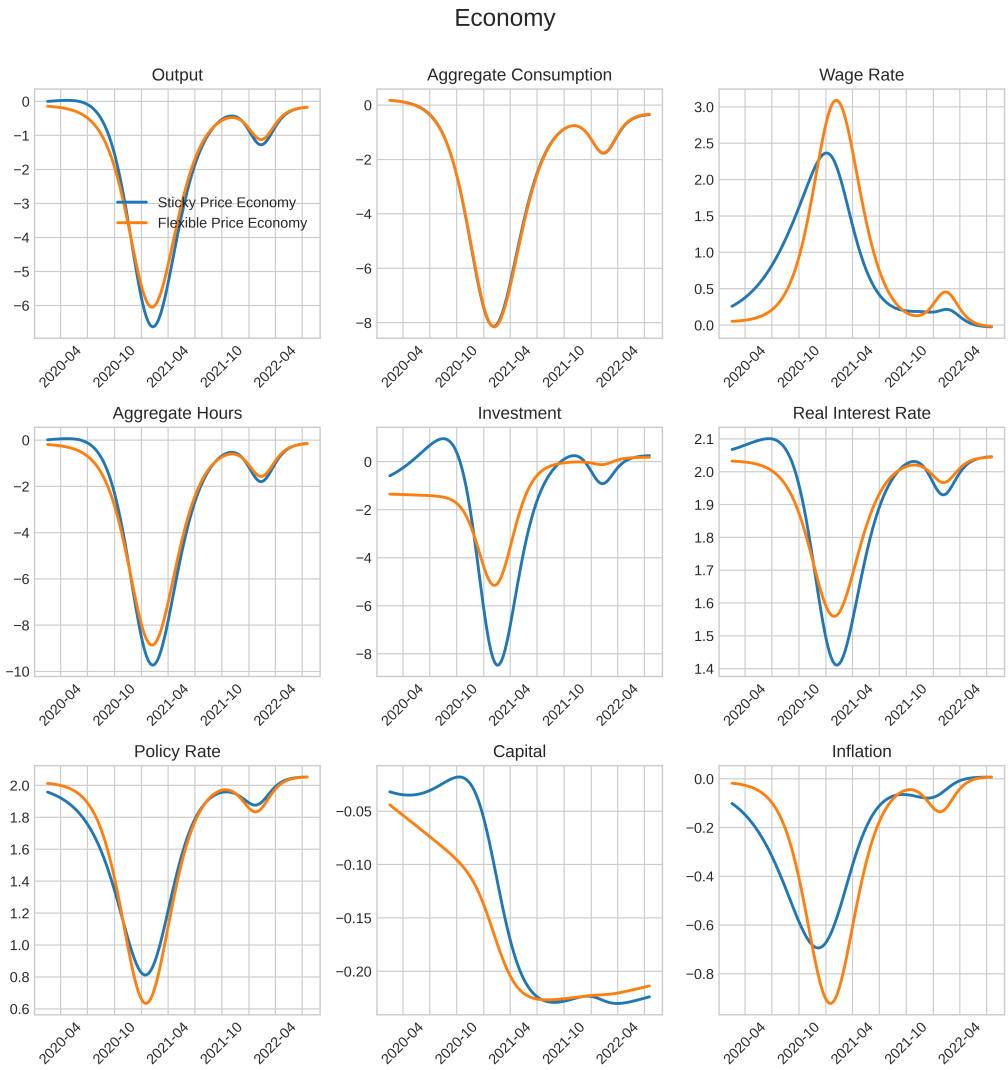

Figure 3: Forecast of Macroeconomic Variables

Acknowledgements

Authors would like to thank Douglas Laxton for initiating this project, Farias Aquiles for his guidance and support, and Kadir Tanyeri for his valuable comments.

References

Botman, D. P., Karam, P. D., Laxton, D., & Rose, D. (2007). *DSGE modeling at the fund: Applications and further developments*. https://www.imf.org/en/Publications/WP/Issues/2016/12/31/DSGE-Modeling-at-the-Fund-Applications-and-Further-Developments-21265

Del Negro, M., Eusepi, S., Giannoni, M. P., Sbordone, A. M., Tambalotti, A., Cocci, M., Hasegawa, R., & Linder, M. (2013). The FRBNY DSGE model. *FRB of New York Staff Report, 647*. https://www.newyorkfed.org/medialibrary/media/research/staff_reports/sr647.pdf

Eichenbaum, M., Rebelo, S., & Trabandt, M. (2020). The macroeconomics of epidemics.

NBER Working Paper, No. 26882. https://doi.org/10.3386/w26882

Fernández-Villaverde, J., & Guerrón-Quintana, P. A. (2021). Estimating DSGE models: Recent advances and future challenges. *Annual Review of Economics*, *13*(1), 229–252. https://doi.org/10.3386/w27715

Goumilevski, A., Farias, A., & Tanyeri, K. (2021). A new macroeconomic modeling framework applied to assess effects of COVID-19. *Proceedings of the 27th International Conference on Computing in Economics and Finance*. https://www.academia.edu/117687479/A_New_Macroeconomic_Modeling_Platform_Applied_to_Assess_Effects_of_COVID_19

Ireland, P. (2004). Technology shocks in the new keynesian model. *NBER Working Paper, No. 10309*. https://www.nber.org/system/files/working_papers/w10309/w10309.pdf

Juillard, M., Laxton, D., McAdam, P., & Pioro, H. (1998). An algorithm competition: First-order iterations versus newton-based technique. *Journal of Economic Dynamics and Control, No. 22, Pp.1291—1318*. https://doi.org/10.1016/S0165-1889(98)00013-X

Smets, F., Christoffel, K., Coenen, G., Motto, R., & Rostagno, M. (2010). DSGE models and their use at the ECB. *SERIEs*, *1*, 51–65. https://doi.org/10.1007/s13209-010-0020-9

Yagihashi, T. (2020). DSGE models used by policymakers: A survey. *Policy Research Institute, Ministry of Finance, Japan, PRI Discussion Paper Series*. https://www.mof.go.jp/pri/research/discussion_paper/ron333.pdf

DRAFT